TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO DỰ ÁN

HỌC PHẦN: XỬ LÝ NGÔN NGỮ TỰ NHIÊN

Đề tài:

INDEX BEAST: HỆ THỐNG HỎI ĐÁP LORE ELDEN RING

Tên nhóm: **NHÓM 4 HOLLOWS**

Sinh viên thực hiện: **Trần Trọng Hoàn**

Nông Hữu Thịnh

[Nguyễn Quang Minh]

[Vũ Ngọc Quý]

Giáo viên hướng dẫn: **[Ph.D Trần Văn Khánh]**

MỤC LỤC

DANH MỤC HÌNH ẢNH **4**

DANH MỤC BẢNG **5**

MỞ ĐẦU **6**

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT **7**

1.1. Tổng quan về xử lý ngôn ngữ tự nhiên và hỏi đáp 7

1.1.1. Khái niệm và vị trí của xử lý ngôn ngữ tự nhiên 7

1.1.2. Bài toán hỏi đáp 7

1.2. Biểu diễn văn bản và truy xuất thông tin 8

1.2.1. Từ tách token đến biểu diễn vector 8

1.2.2. Tìm kiếm từ khóa và BM25 9

1.2.3. Truy xuất lai và hợp nhất thứ hạng 9

1.3. Xếp hạng lại kết quả truy xuất 9

1.4. Mô hình ngôn ngữ lớn và sinh văn bản có truy xuất 10

1.4.1. Mô hình ngôn ngữ lớn 10

1.4.2. Sinh văn bản có truy xuất 11

1.4.3. Ảo giác và trích dẫn nguồn 11

1.5. Chuẩn bị dữ liệu văn bản 12

1.5.1. Thu thập và làm sạch 12

1.5.2. Chia đoạn 12

1.6. Đánh giá hệ thống truy xuất và hỏi đáp 13

1.6.1. Chỉ số truy xuất 13

1.6.2. Đánh giá câu trả lời 13

1.6.3. Bộ dữ liệu đánh giá và độ tin cậy 13

1.7. Quy trình thực hiện và trách nhiệm sử dụng 14

CHƯƠNG 2. BÀI TOÁN, PHƯƠNG PHÁP VÀ MÔ HÌNH **15**

2.1. Phát biểu bài toán 15

2.2. Mục tiêu và tiêu chí đánh giá 15
2.3. Nguồn dữ liệu và từ điển dữ liệu 16
2.4. Quy mô và phân bố dữ liệu thu thập 17
2.5. Quy trình thu thập và xử lý dữ liệu ngoại tuyến 19
2.6. Làm sạch và chia đoạn 19
2.7. Chất lượng dữ liệu và giới hạn 21
2.8. Lựa chọn thành phần mô hình 22
2.9. Thiết kế hệ thống 23
2.10. Quy trình xử lý một câu hỏi 24
2.11. Thiết kế đánh giá 26
2.12. Quy trình phát triển và bảo đảm chất lượng 27
CHƯƠNG 3. THỬ NGHIỆM VÀ ĐÁNH GIÁ **28**
3.1. Thiết lập thực nghiệm 28
3.2. Kết quả truy xuất 29
3.3. Đánh giá câu trả lời 30
3.4. Phân tích lỗi và khả năng khái quát hóa 32
3.5. Kiểm thử kỹ thuật và chất lượng phần mềm 32
3.6. Chương trình minh họa 33
3.7. Kết luận và hướng phát triển 34
TÀI LIỆU THAM KHẢO **36**

DANH MỤC HÌNH ẢNH

1 Số trang ứng viên theo danh mục khi khảo sát nguồn 18
2 Quy trình dữ liệu ngoại tuyến gồm năm bước 19
3 Kiến trúc tổng thể của Index Beast 23
4 Luồng xử lý một câu hỏi của hệ thống (thiết kế mục tiêu) 25
5 Nghịch đảo thứ hạng (RR) của tài liệu đúng đầu tiên cho từng câu hỏi 30

DANH MỤC BẢNG

1 Mục tiêu học máy và tiêu chí theo dõi 16
2 Nguồn dữ liệu và kết quả khảo sát 16
3 Từ điển dữ liệu của một đoạn đã chia (thiết kế) 17
4 Số trang ứng viên theo danh mục 17
5 Kết quả thu thập tập mẫu 18
6 Số cửa sổ chia đoạn theo giới hạn token trên mẫu 20 trang 20
7 Vấn đề chất lượng dữ liệu và quyết định xử lý 21
8 Lựa chọn thành phần của quy trình RAG 22
9 Công nghệ chính và phiên bản 24
10 Cơ cấu bộ câu hỏi chuẩn phiên bản 0 26
11 Kế hoạch sáu sprint 27
12 Cấu hình thí nghiệm cơ sở 28
13 Kết quả truy xuất và sinh trả lời của thí nghiệm cơ sở 29
14 Điểm và nhận xét theo từng câu hỏi (thang 0–2) 31
15 Kết quả kiểm tra phần mềm trên nhánh phát triển 33
16 Các thành phần minh họa và trạng thái 34

MỞ ĐẦU

Các trò chơi nhập vai có thế giới truyện phức tạp thường khiến người chơi phải tra cứu nhiều trang wiki để hiểu một nhân vật hoặc một sự kiện. Với Elden Ring, phần lớn cốt truyện được kể gián tiếp qua mô tả vật phẩm, lời thoại và địa điểm, nên một câu hỏi đơn giản cũng có thể cần ghép thông tin từ nhiều trang. Các mô hình ngôn ngữ lớn trả lời trôi chảy nhưng có thể đưa ra chi tiết không có trong nguồn. Vì vậy, đề tài kết hợp truy xuất thông tin với mô hình ngôn ngữ để câu trả lời bám vào tài liệu cụ thể và có trích dẫn.

Đề tài xây dựng Index Beast, một hệ thống hỏi đáp về lore Elden Ring bản gốc (base game) bằng tiếng Việt và tiếng Anh, dùng chung một phần phụ trợ cho website và bot Discord. Hệ thống dựa trên kiến trúc sinh văn bản có truy xuất (Retrieval-Augmented Generation, RAG): câu hỏi được dùng để tìm các đoạn văn liên quan trong kho tri thức, sau đó mô hình ngôn ngữ viết câu trả lời dựa trên các đoạn đó. Việc tìm kiếm kết hợp biểu diễn vector ngữ nghĩa với tìm kiếm từ khóa, rồi hợp nhất và xếp hạng lại kết quả.

Mục tiêu của báo cáo là trình bày cơ sở lý thuyết của các thành phần, phát biểu bài toán, mô tả dữ liệu và thiết kế hệ thống, đồng thời ghi lại kết quả của giai đoạn đầu (Sprint 1 trong kế hoạch sáu sprint). Giai đoạn này mới hoàn thành nền tảng kỹ thuật, quy trình thu thập dữ liệu, bộ câu hỏi đánh giá và một thí nghiệm cơ sở trên tập con dữ liệu. Báo cáo không khẳng định chất lượng của hệ thống hoàn chỉnh; các số liệu được nêu cùng điều kiện và giới hạn của từng lần đo.

Báo cáo gồm ba chương. Chương 1 trình bày cơ sở lý thuyết về xử lý ngôn ngữ tự nhiên, truy xuất thông tin, mô hình ngôn ngữ lớn và đánh giá. Chương 2 trình bày bài toán, dữ liệu, phương pháp và thiết kế hệ thống. Chương 3 trình bày thử nghiệm, kết quả, phân tích và hướng phát triển.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

1.1. Tổng quan về xử lý ngôn ngữ tự nhiên và hỏi đáp

1.1.1. Khái niệm và vị trí của xử lý ngôn ngữ tự nhiên

Xử lý ngôn ngữ tự nhiên (Natural Language Processing, NLP) là lĩnh vực nghiên cứu cách để máy tính phân tích, biểu diễn và tạo ra ngôn ngữ của con người. Các nhiệm vụ điển hình gồm phân loại văn bản, gán nhãn từ loại, nhận diện thực thể, dịch máy, tóm tắt, tìm kiếm thông tin và hỏi đáp. Mỗi nhiệm vụ có dữ liệu, thước đo và giả định khác nhau, nhưng đều phải đối mặt với những đặc điểm của ngôn ngữ như tính nhập nhằng, phụ thuộc ngữ cảnh và sự đa dạng cách diễn đạt cho cùng một ý [1].

Trong vài năm gần đây, các mô hình dựa trên kiến trúc Transformer được huấn luyện trước trên lượng văn bản rất lớn đã trở thành nền tảng của hầu hết các hệ thống NLP. Chúng có hai vai trò chính trong đề tài này. Vai trò thứ nhất là mô hình biểu diễn, biến một đoạn văn bản thành vector để so sánh ý nghĩa. Vai trò thứ hai là mô hình sinh, viết văn bản mới từ một chỉ dẫn và ngữ cảnh. Hai vai trò này được kết nối qua bước truy xuất thông tin, vốn là một lĩnh vực có lịch sử lâu đời trong khoa học máy tính [2].

1.1.2. Bài toán hỏi đáp

Hỏi đáp (Question Answering) là bài toán trả lời một câu hỏi bằng ngôn ngữ tự nhiên. Có thể phân biệt hai cách tiếp cận. Với hỏi đáp đóng (closed-book), mô hình trả lời chỉ dựa trên kiến thức đã học trong tham số. Cách này đơn giản nhưng khó cập nhật, khó chỉ ra nguồn và dễ đưa ra chi tiết sai khi gặp chủ đề hiếm. Với hỏi đáp dựa trên tài liệu (open-book), hệ thống tìm các đoạn văn liên quan trong một kho tri thức rồi mới trả lời từ các đoạn đó. Cách này cho phép kiểm tra câu trả lời đối chiếu với nguồn [7].

Bài toán hỏi đáp về một thế giới hư cấu có một số đặc điểm riêng. Kiến thức nằm ở những trang wiki do cộng đồng biên soạn, có thể thay đổi theo thời gian. Một nhân vật thường có nhiều tên gọi hoặc danh hiệu. Những chi tiết mơ hồ có thể là suy đoán của người chơi chứ

không phải nội dung chính thức. Hệ thống cần phân biệt những điều đó khi trình bày, và cần từ chối các câu hỏi nằm ngoài phạm vi thay vì trả lời bằng kiến thức chung.

1.2. Biểu diễn văn bản và truy xuất thông tin

1.2.1. Từ tách token đến biểu diễn vector

Mô hình ngôn ngữ không xử lý chuỗi ký tự trực tiếp. Văn bản được tách thành các token, thường là các đơn vị con của từ, rồi ánh xạ thành vector. Số token của một đoạn văn là đại lượng quan trọng vì mô hình biểu diễn và mô hình sinh đều có giới hạn độ dài đầu vào. Độ dài tính theo ký tự hoặc theo từ chỉ là ước lượng; muốn kiểm tra giới hạn một cách chính xác cần đếm bằng đúng tokenizer của mô hình sẽ dùng.

Mô hình embedding ánh xạ một đoạn văn thành một vector có số chiều cố định sao cho các đoạn gần nghĩa nằm gần nhau. Độ gần thường được đo bằng độ tương đồng cosine:

$$\cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \, \|\mathbf{d}\|}.$$

Họ mô hình E5 được huấn luyện bằng học tương phản trên các cặp văn bản liên quan [4]. Một đặc điểm cần lưu ý khi dùng là mô hình yêu cầu thêm tiền tố `query:` cho câu hỏi và `passage:` cho đoạn tài liệu; bỏ qua hoặc thêm hai lần tiền tố làm vector khác với cách mô hình được huấn luyện [4, 16]. Phiên bản đa ngôn ngữ hỗ trợ nhiều ngôn ngữ, trong đó có tiếng Việt, nên một câu hỏi tiếng Việt vẫn có thể được so sánh với tài liệu tiếng Anh, nhưng chất lượng của việc so sánh chéo ngôn ngữ cần được kiểm chứng trên dữ liệu thực tế.

Tìm kiếm theo vector (dense retrieval) có ưu điểm là nắm bắt được các cách diễn đạt khác nhau của cùng một ý. Nhược điểm là có thể bỏ sót tên riêng hiếm hoặc từ khóa chính xác, vì các tên này ít được biểu diễn tốt trong không gian vector.

1.2.2. Tìm kiếm từ khóa và BM25

Tìm kiếm từ khóa xếp hạng tài liệu theo mức độ trùng khớp từ giữa truy vấn và tài liệu. BM25 là hàm xếp hạng được dùng rộng rãi, thuộc họ mô hình xác suất [3]. Điểm của tài liệu d với truy vấn q là

$$\text{BM25}(q,d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1 \left(1 - b + b\,\dfrac{|d|}{\text{avgdl}}\right)},$$

trong đó $f(t,d)$ là số lần từ t xuất hiện trong d, $|d|$ là độ dài tài liệu, avgdl là độ dài trung bình, $\text{IDF}(t)$ phản ánh độ hiếm của từ trong toàn kho, còn k_1 và b là hai tham số điều chỉnh mức bão hòa tần suất và chuẩn hóa độ dài. Từ hiếm có trọng số cao, nên BM25 thể hiện tốt với tên nhân vật hoặc địa danh. Điểm yếu của nó là không nhận ra từ đồng nghĩa và phụ thuộc vào việc truy vấn và tài liệu dùng cùng ngôn ngữ, nên với kho tài liệu tiếng Anh, truy vấn tiếng Việt cần được chuyển sang tiếng Anh trước khi tìm.

1.2.3. Truy xuất lai và hợp nhất thứ hạng

Hai cách tìm kiếm trên bổ sung cho nhau: vector bắt được nghĩa, từ khóa bắt được tên chính xác. Truy xuất lai (hybrid retrieval) chạy cả hai rồi hợp nhất kết quả. Điểm của hai hệ thống không cùng thang đo nên việc cộng trực tiếp là không hợp lý. Reciprocal Rank Fusion (RRF) tránh vấn đề này bằng cách chỉ dùng thứ hạng [5]:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)},$$

với R là tập các danh sách xếp hạng, $\text{rank}_r(d)$ là vị trí của d trong danh sách r và k là hằng số làm giảm ảnh hưởng của những vị trí đầu. Tài liệu xuất hiện cao trong cả hai danh sách được ưu tiên. Qdrant hỗ trợ truy vấn nhiều giai đoạn, cho phép chạy đồng thời tìm kiếm dense và sparse rồi hợp nhất bằng RRF ngay trên máy chủ trong một lần gọi [9].

1.3. Xếp hạng lại kết quả truy xuất

Các mô hình embedding mã hóa câu hỏi và tài liệu một cách độc lập, nên có thể tính trước vector của toàn bộ kho và tìm kiếm nhanh. Đổi lại, chúng không xét tương tác chi tiết giữa câu

hỏi và từng đoạn. Mô hình xếp hạng lại (reranker) kiểu cross-encoder đưa cặp (câu hỏi, đoạn văn) vào cùng một mô hình và cho ra điểm liên quan trực tiếp [6]. Cách này chính xác hơn nhưng tốn tính toán hơn nhiều, vì phải chạy mô hình cho từng cặp. Thực tế thường áp dụng hai giai đoạn: truy xuất nhanh lấy một số lượng ứng viên vừa phải (chẳng hạn vài chục), rồi reranker chọn ra một số ít đoạn tốt nhất để đưa cho mô hình sinh. Độ trễ của giai đoạn này phụ thuộc vào số ứng viên, độ dài đoạn và phần cứng, nên cần được đo trên môi trường triển khai thay vì suy ra từ lý thuyết.

Điểm của reranker cũng được dùng để quyết định hệ thống có đủ bằng chứng hay không. Nếu điểm cao nhất thấp hơn một ngưỡng, hệ thống nên thông báo chưa tìm thấy thông tin thay vì bắt mô hình sinh trả lời từ những đoạn không liên quan. Ngưỡng này không có giá trị đúng sẵn mà cần được hiệu chỉnh trên một bộ dữ liệu đánh giá.

1.4. Mô hình ngôn ngữ lớn và sinh văn bản có truy xuất

1.4.1. Mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn (Large Language Model, LLM) là mô hình Transformer có số tham số rất lớn, được huấn luyện để dự đoán token kế tiếp, sau đó thường được tinh chỉnh để làm theo chỉ dẫn. Khi sinh văn bản, mô hình chọn từng token dựa trên phân phối xác suất; tham số như nhiệt độ (temperature) điều chỉnh mức ngẫu nhiên của việc chọn này. Một số mô hình có chế độ suy luận (thinking), trong đó mô hình sinh thêm một chuỗi lập luận trước câu trả lời cuối cùng để cải thiện độ chính xác trên bài toán phức tạp, đổi lại tốn thêm thời gian và token. Ở chế độ này, một số tham số lấy mẫu có thể bị bỏ qua [10]. Với bài toán trả lời từ ngữ cảnh đã cho, chế độ suy luận không phải lúc nào cũng cần thiết.

Mô hình có thể được truy cập qua API theo hình thức tính tiền theo số token vào và ra. Chi phí vì vậy phụ thuộc vào độ dài ngữ cảnh đưa vào mỗi lượt, và có thể giảm khi phần đầu của lời nhắc lặp lại giữa các lượt gọi được bộ nhớ đệm (prompt caching) của nhà cung cấp tính giá thấp hơn [10].

1.4.2. Sinh văn bản có truy xuất

Sinh văn bản có truy xuất (RAG) kết hợp bộ nhớ trong tham số của mô hình với bộ nhớ ngoài là một kho tài liệu [7]. Quy trình cơ bản gồm bốn bước: biến câu hỏi thành truy vấn, truy xuất các đoạn liên quan, ghép các đoạn vào lời nhắc cùng câu hỏi, rồi để mô hình sinh câu trả lời dựa trên các đoạn đó. So với việc chỉ dùng mô hình, RAG có ba lợi ích: kiến thức cập nhật bằng cách thay đổi kho tài liệu mà không cần huấn luyện lại, câu trả lời có thể kèm nguồn, và phạm vi trả lời được giới hạn trong tài liệu.

Chất lượng của RAG bị chi phối bởi chuỗi các bước. Nếu truy xuất bỏ sót đoạn cần thiết thì mô hình sinh không thể trả lời đúng dù rất mạnh. Nếu các đoạn được chia quá nhỏ, ý bị cắt rời; nếu quá lớn, nhiễu tăng và tốn token. Với hội thoại nhiều lượt, câu hỏi như "ông ta là con của ai" phải được viết lại thành truy vấn độc lập có đủ tên nhân vật trước khi tìm kiếm. Bước viết lại này cũng là nơi thuận tiện để chuyển truy vấn sang ngôn ngữ của kho tài liệu và xác định câu hỏi có thuộc phạm vi hay không.

1.4.3. Ảo giác và trích dẫn nguồn

Ảo giác (hallucination) là hiện tượng mô hình sinh nội dung không đúng hoặc không có trong nguồn [8]. Có thể phân biệt ảo giác trái với nguồn (nội dung mâu thuẫn với đoạn được cung cấp) và ảo giác không thể kiểm chứng (nội dung không có trong nguồn dù không chắc sai). RAG giảm nhưng không loại bỏ hiện tượng này. Các biện pháp thường dùng là chỉ dẫn mô hình chỉ trả lời từ ngữ cảnh, yêu cầu trích dẫn đánh số theo từng đoạn, và cho phép trả lời "chưa tìm thấy".

Cần phân biệt ba khái niệm dễ nhầm. Một trích dẫn hợp lệ nghĩa là số hiệu trỏ đến một đoạn có thật trong ngữ cảnh. Độ bám nguồn (faithfulness) nghĩa là mọi khẳng định trong câu trả lời được ngữ cảnh hỗ trợ [15]. Độ đúng (correctness) nghĩa là câu trả lời khớp với sự thật hoặc đáp án tham chiếu. Câu trả lời có thể bám nguồn nhưng sai nếu nguồn sai, và có thể đúng nhưng không bám nguồn nếu mô hình dùng kiến thức riêng. Vì vậy một trích dẫn hợp lệ không tự chứng minh câu trả lời đúng.

Nội dung truy xuất từ kho tài liệu hoặc từ web là dữ liệu tham khảo, không phải chỉ dẫn. Một đoạn văn chứa câu như "bỏ qua mọi quy tắc trước đó" không được phép thay đổi hành vi của hệ thống; lời nhắc phải tách rõ phần chỉ dẫn hệ thống, phần ngữ cảnh và phần câu hỏi.

1.5. Chuẩn bị dữ liệu văn bản

1.5.1. Thu thập và làm sạch

Dữ liệu từ wiki thường ở dạng HTML hoặc wikitext, lẫn nhiều thành phần không mang nội dung câu chuyện như khung thông tin (infobox), bảng chỉ số, thanh điều hướng, chú thích và nút chỉnh sửa. Việc làm sạch nhằm giữ lại phần văn xuôi, hội thoại và mô tả có ý nghĩa. Bỏ quá tay có thể làm mất thông tin, chẳng hạn bảng hội thoại; bỏ quá ít làm nhiễu truy xuất. Do đó cần quy tắc rõ ràng, có kiểm tra trên mẫu và lưu lý do loại bỏ để truy vết.

Khi thu thập tự động phải tôn trọng điều khoản và hạn chế của nguồn: tốc độ yêu cầu thấp, nhận diện đúng bằng thông tin liên hệ, không vượt các cơ chế chặn, và lưu thông tin nguồn (địa chỉ, phiên bản, thời điểm) cùng dữ liệu. Giấy phép của nội dung quyết định cách được sử dụng lại; nội dung theo giấy phép CC BY-SA 4.0 cho phép chia sẻ và chuyển thể nếu ghi công tác giả, nêu rõ thay đổi và giữ cùng giấy phép khi phân phối [12].

1.5.2. Chia đoạn

Chia đoạn (chunking) là việc cắt tài liệu thành các đơn vị được đánh chỉ mục. Kích thước đoạn là sự đánh đổi: đoạn nhỏ cho độ chính xác truy xuất cao nhưng dễ thiếu ngữ cảnh, đoạn lớn giữ ngữ cảnh nhưng pha loãng vector và bị giới hạn bởi độ dài tối đa của mô hình embedding. Hai kỹ thuật phổ biến là chia theo cấu trúc (tiêu đề, đoạn, câu) và chia theo cửa sổ token có phần chồng lấn (overlap) để một ý nằm ở ranh giới không bị cắt mất. Mỗi đoạn nên giữ tiêu đề trang và đường dẫn các tiêu đề cha, vì một câu như "bà ấy sinh ra đã mang bệnh" chỉ có nghĩa khi biết đang nói về nhân vật nào. Kích thước tối ưu phụ thuộc dữ liệu và câu hỏi, nên giá trị khởi đầu cần được kiểm chứng bằng kết quả truy xuất.

1.6. Đánh giá hệ thống truy xuất và hỏi đáp

1.6.1. Chỉ số truy xuất

Với mỗi câu hỏi có tập tài liệu liên quan đã biết, chất lượng truy xuất được đo trên k kết quả đầu. Hit@k là tỉ lệ câu hỏi có ít nhất một tài liệu liên quan trong k kết quả đầu. Recall@k là tỉ lệ tài liệu liên quan tìm được trong k kết quả đầu, tính trung bình theo câu hỏi. Mean Reciprocal Rank (MRR@k) xét thêm vị trí của tài liệu liên quan đầu tiên:

$$\text{MRR@}k = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{\text{rank}_i},$$

với rank_i là vị trí của tài liệu liên quan đầu tiên của câu thứ i và số hạng bằng 0 nếu không có trong k kết quả đầu [2]. Hit@k cao nhưng MRR thấp cho thấy tài liệu đúng có mặt nhưng chưa được xếp lên đầu, tức là còn chỗ cho bước xếp hạng lại.

1.6.2. Đánh giá câu trả lời

Chất lượng câu trả lời được xét theo hai tiêu chí chính là độ đúng và độ bám nguồn, có thể chấm theo thang điểm rời rạc, chẳng hạn 0–2, với quy ước rõ cho từng mức. Với những câu hỏi nằm ngoài phạm vi, tiêu chí là tỉ lệ từ chối đúng thay vì độ đúng. Việc chấm có thể do người thực hiện hoặc do một mô hình ngôn ngữ khác đảm nhiệm. Chấm bằng mô hình có chi phí thấp và nhất quán nhưng có thể thiên lệch, nên cần đối chiếu với một mẫu do người chấm để xác nhận [15]. Cần ghi rõ ai hoặc cái gì đã chấm khi báo cáo điểm.

1.6.3. Bộ dữ liệu đánh giá và độ tin cậy

Bộ câu hỏi chuẩn (golden set) gồm câu hỏi, đáp án tham chiếu, nguồn tham chiếu, nhóm câu hỏi và độ khó. Các điểm cần bảo đảm là: câu hỏi đa dạng về loại và độ khó, có cả câu ngoài phạm vi, đáp án được người khác kiểm tra chéo, và đáp án không được đưa vào chỉ mục hay lời nhắc của hệ thống. Bộ dữ liệu nhỏ cho ước lượng có độ biến thiên lớn; ví dụ một bộ 10 câu thay đổi 10 điểm phần trăm chỉ vì một câu. Nếu bộ dữ liệu được dùng để chỉnh lời nhắc hoặc

tham số, nó không còn là phép thử độc lập; cần giữ riêng một tập chưa dùng để chỉnh. Kết quả cũng chỉ có ý nghĩa với đúng phạm vi dữ liệu đã đưa vào chỉ mục.

1.7. Quy trình thực hiện và trách nhiệm sử dụng

Một dự án hỏi đáp dựa trên RAG có thể được tổ chức thành các giai đoạn: xác định phạm vi và yêu cầu, thu thập dữ liệu, làm sạch và chia đoạn, lập chỉ mục, xây dựng quy trình truy xuất và sinh, đánh giá, rồi triển khai. Các giai đoạn lặp lại khi có bằng chứng mới, và mỗi thay đổi nên gắn với một vấn đề đã quan sát. Để người khác chạy lại được kết quả, cần lưu phiên bản dữ liệu, mã nguồn, cấu hình, mô hình và đầu ra của lần chạy được dùng trong báo cáo.

Trách nhiệm sử dụng gồm ghi nguồn và giấy phép của nội dung và hình ảnh, không đưa khóa truy cập hoặc dữ liệu cá nhân vào mã nguồn và bản ghi, giới hạn chi phí và tần suất gọi dịch vụ bên ngoài, và nêu rõ cho người dùng rằng câu trả lời có thể sai và cần đối chiếu với nguồn. Với một dự án hâm mộ, cần thông báo rõ rằng sản phẩm không liên kết với nhà phát hành trò chơi.

CHƯƠNG 2. BÀI TOÁN, PHƯƠNG PHÁP VÀ MÔ HÌNH

2.1. Phát biểu bài toán

Index Beast nhận một câu hỏi bằng tiếng Việt hoặc tiếng Anh về lore Elden Ring base game, kèm lịch sử hội thoại nếu có, và trả về một câu trả lời bằng ngôn ngữ của người hỏi. Câu trả lời phải dựa trên các đoạn tài liệu truy xuất được, gắn trích dẫn đánh số về tên trang, mục và địa chỉ nguồn, và phải nói rõ khi thông tin chỉ là suy đoán của cộng đồng hoặc khi không tìm thấy. Đầu vào và đầu ra có thể mô tả như sau.

- **Đầu vào:** câu hỏi, kênh gọi (web hoặc Discord), mã hội thoại, tùy chọn ngôn ngữ và bật hoặc tắt tìm kiếm web dự phòng.
- **Đầu ra:** văn bản trả lời, danh sách nguồn (chỉ số, tiêu đề, mục, địa chỉ, đoạn trích, nguồn kho hay nguồn web), cờ cho biết có đủ bằng chứng hay không và độ trễ.
- **Từ chối:** câu hỏi ngoài phạm vi (hướng dẫn xây dựng nhân vật, chỉ số vũ khí, trò chơi khác, yêu cầu không liên quan) và câu hỏi chưa có đủ bằng chứng.

Phạm vi nội dung được chốt tại ADR-004 là lore của bản gốc Elden Ring; các bản mở rộng, kể cả Shadow of the Erdtree, và trò chơi Elden Ring Nightreign nằm ngoài phạm vi. Quyết định này giúp nhóm tập trung vào xây dựng và đánh giá quy trình RAG thay vì xử lý các nguồn lẫn nội dung. Về điều kiện triển khai, hệ thống chạy trên một máy chủ ảo 4 vCPU và 8 GB RAM không có GPU, gọi mô hình sinh qua API, phục vụ khoảng vài trăm người dùng với tối đa khoảng 50 người đồng thời.

2.2. Mục tiêu và tiêu chí đánh giá

Mục tiêu và ngưỡng dưới đây lấy từ đặc tả yêu cầu của dự án (SRS) và tài liệu công nghệ. Các ngưỡng là giá trị mục tiêu ban đầu và sẽ được hiệu chỉnh khi có số đo thực tế. Cột cuối nêu trạng thái tại thời điểm viết báo cáo, để tránh nhầm giữa mục tiêu và kết quả.

Bảng 1. Mục tiêu học máy và tiêu chí theo dõi

Mục tiêu	Tiêu chí	Trạng thái hiện tại
Truy xuất đúng tài liệu	Recall@5 $\geq$ 0,80 trên bộ câu hỏi chuẩn (NFR-QUAL-01)	Chưa đo trên bộ chuẩn. Thí nghiệm cơ sở: Hit@5 = 10/10 trên 10 câu và tập con 240 bản ghi (mục 3.2)
Câu trả lời đúng và bám nguồn	Correctness $\geq$ 70%, faithfulness $\geq$ 85% (NFR-QUAL-02)	Chưa đo trên bộ chuẩn. Thí nghiệm cơ sở: 20/20 điểm correctness và 20/20 điểm faithfulness trên 10 câu
Từ chối đúng câu ngoài phạm vi	Tỉ lệ từ chối đúng riêng, tách khỏi Recall và MRR	Chưa đo; bộ chuẩn có 6 câu ngoài phạm vi
Độ trễ chấp nhận được	Token đầu $\leq$ 3,5 s, trả lời đầy đủ $\leq$ 10 s (p95); xếp hạng lại 30 đoạn $\leq$ 800 ms	Chưa đo; quy trình hoàn chỉnh chưa được nối
Chi phí kiểm soát được	Chi phí gọi mô hình $\leq$ 30 USD mỗi tháng ở mức 1.000 câu mỗi ngày (NFR-COST-01)	Ước tính từ bảng giá (mục 2.8), chưa đo trên lưu lượng thật

2.3. Nguồn dữ liệu và từ điển dữ liệu

Nguồn chính là Elden Ring Wiki trên wiki.gg [13]. Nội dung của wiki này theo giấy phép CC BY-SA 4.0 [12], nên mọi câu trả lời cần ghi nguồn và địa chỉ trang. Wiki chạy trên MediaWiki và cung cấp giao diện lập trình cho phép liệt kê trang theo danh mục và lấy HTML kèm số phiên bản (revision) của trang [11]. Nhóm đã khảo sát thêm hai nguồn khác; kết quả được tóm tắt ở Bảng 2.

Bảng 2. Nguồn dữ liệu và kết quả khảo sát

Nguồn	Vai trò	Kết quả kiểm chứng
wiki.gg	Nguồn thô chính	Liệt kê theo danh mục, lấy nội dung (parse) và thông tin ảnh đều hoạt động; footer ghi CC BY-SA 4.0
fanapis	Khảo sát dữ liệu có cấu trúc để xây bảng thực thể	7 loại tài nguyên truy cập được, mỗi loại thử hai trang kết quả; số bản ghi: boss 106, NPC 55, địa điểm 177, vật phẩm 462, vũ khí 307, bùa 87, linh thể 64. Chưa xác minh giấy phép tái phân phối
Fandom	Dự kiến đối chiếu 10 trang	Tệp robots.txt trả HTTP 403 nên dừng, không tải bài; chưa có so sánh nội dung hợp lệ

Dump của wiki	Phương án thu thập khác	Trang yêu cầu dump đọc được nhưng bản hiện hành và bản đầy đủ lịch sử đều chưa được tạo; không có đường dẫn tải

Dữ liệu thô của mỗi trang gồm tiêu đề, địa chỉ, số phiên bản, danh sách danh mục, HTML của nội dung, mã băm và thời điểm tải. Sau khi làm sạch và chia đoạn, mỗi đoạn dự kiến mang siêu dữ liệu ở Bảng 3; các trường này phục vụ truy vết nguồn, lọc phạm vi và đánh giá.

Bảng 3. Từ điển dữ liệu của một đoạn đã chia (thiết kế)

Trường	**Ý nghĩa**	**Ghi chú**
`doc_id`, `title`, `url`	Trang chứa đoạn, tiêu đề và địa chỉ	Dùng để trích dẫn
`section`	Đường dẫn các tiêu đề cha và con	Giữ ngữ cảnh khi đoạn bị cắt
`source`	Nguồn dữ liệu (wiki.gg, …)	Hỗ trợ nhiều nguồn
`game_scope`	Phạm vi trò chơi	Chỉ gán base sau khi rà soát từng mục
`source_type`	Văn bản trong game, văn xuôi wiki hoặc suy đoán	Để phân biệt khi trả lời
`lang`, `updated_at`	Ngôn ngữ và thời điểm cập nhật	Hỗ trợ cập nhật tăng dần
`text`, `token_count`	Nội dung và số token	Số token đếm bằng tokenizer của mô hình embedding

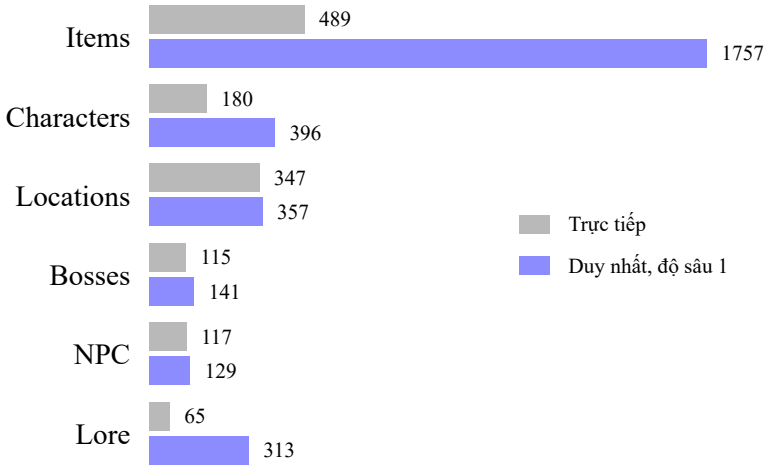
2.4. Quy mô và phân bố dữ liệu thu thập

Bảng 4 cho số trang ứng viên theo danh mục khi khảo sát, đếm theo không gian tên bài viết. Cột thứ hai chỉ tính trang nằm trực tiếp trong danh mục; cột thứ ba gộp thêm danh mục con ở độ sâu một và đã loại trùng. Các nhóm giao nhau nên không cộng các hàng để suy ra số trang duy nhất. Sau khi khử trùng, khảo sát thu được 2.602 trang ứng viên, trong đó có thể lẫn nội dung bản mở rộng.

Bảng 4. Số trang ứng viên theo danh mục

Danh mục	**Trang trực tiếp**	**Danh mục con**	**Duy nhất, độ sâu 1**
NPC	117	3	129
Bosses	115	3	141
Locations	347	17	357
Lore	65	9	313
Items	489	19	1.757

Characters	180	10	396



Hình 1. Số trang ứng viên theo danh mục khi khảo sát nguồn

Danh mục Items có nhiều trang ứng viên nhất. Danh mục Lore tăng từ 65 lên 313 trang khi gộp danh mục con, và Characters tăng từ 180 lên 396, cho thấy nhiều trang nằm ở các danh mục con. Vì vậy việc chọn nguồn theo một danh mục duy nhất, hoặc chỉ đếm trang trực tiếp, sẽ bỏ sót nhiều trang. Cũng cần lưu ý đây là số ứng viên trước khi rà soát phạm vi và nội dung.

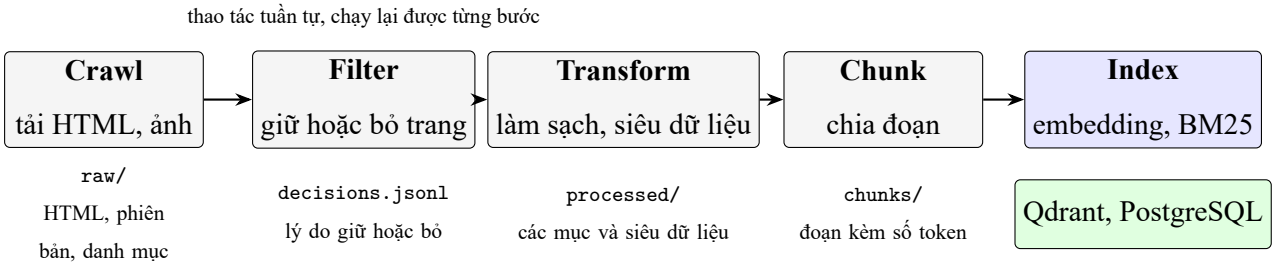
Từ các ứng viên trên, crawler đã tải một tập mẫu 200 trang, chọn theo thứ tự khám phá cân bằng giữa các danh mục, đồng thời tải ảnh liên quan. Bảng 5 nêu số liệu của lượt thu thập này.

Bảng 5. Kết quả thu thập tập mẫu

Chỉ tiêu	**Giá trị**
Số trang duy nhất, số bản ghi thô	200 trang, đủ 200 tệp thô và tệp liệt kê ảnh
Ứng viên bị loại theo không gian tên, trang chuyển hướng hoặc phạm vi	51
Ảnh duy nhất sau khử trùng theo mã băm	1.188 tệp, khoảng 1,67 GB, sau khi loại biểu tượng giao diện
Số lần xuất hiện ảnh trong tập mẫu	1.328
Số yêu cầu mạng và thời gian	1.391 yêu cầu, khoảng 24 phút
Kiểm tra toàn vẹn	Mã băm tệp thô, ảnh và số phiên bản đều khớp; báo cáo nghiệm thu không có lỗi

2.5. Quy trình thu thập và xử lý dữ liệu ngoại tuyến

Việc chuẩn bị dữ liệu được tách khỏi dịch vụ trực tuyến và chia thành năm bước độc lập (Hình 2). Mỗi bước đọc kết quả của bước trước từ thư mục dữ liệu, ghi kết quả của mình, và có thể chạy lại riêng. Nhờ vậy khi đổi kích thước đoạn chỉ cần chia đoạn và lập chỉ mục lại mà không phải tải lại dữ liệu, và khi gặp một câu trả lời sai có thể mở từng tệp trung gian để xem sai ở bước nào.



Hình 2. Quy trình dữ liệu ngoại tuyến gồm năm bước

Chính sách thu thập nhằm không gây tải cho nguồn. Trình thu thập chạy tuần tự với khoảng cách tối thiểu một giây giữa mọi yêu cầu, kể cả yêu cầu tệp robots và ảnh; thử lại tối đa hai lần với thời gian chờ tăng dần, tôn trọng tiêu đề Retry-After và dừng nếu phải chờ quá 60 giây; chỉ dùng HTTPS với danh sách máy chủ cho phép; và không vượt các cơ chế chặn. Phản hồi bài viết tối đa 8 MB, ảnh tối đa 20 MB và 40 triệu điểm ảnh. Định dạng ảnh được kiểm tra bằng thư viện xử lý ảnh trên dữ liệu thực thay vì tin vào tiêu đề MIME. Địa chỉ email liên hệ đặt trong tệp cấu hình môi trường và không ghi vào báo cáo hoặc mã nguồn.

Thu thập được thiết kế theo hướng cập nhật tăng dần: bước tải đã kiểm tra số phiên bản và mã băm để dùng lại dữ liệu sẵn có. Ở bước lập chỉ mục, dự kiến chỉ trang có mã băm nội dung đổi mới được ghi lại và các đoạn của trang bị xóa được dọn khỏi chỉ mục; phần này chưa được triển khai. Mã nguồn dùng giao diện nguồn dữ liệu chung, nên thêm một nguồn mới chỉ cần thêm một mô-đun và một mục cấu hình.

2.6. Làm sạch và chia đoạn

Làm sạch nhằm bỏ phần không mang nội dung kể chuyện: khung thông tin chỉ số, bảng điều hướng, chú thích, mục lục, nút chỉnh sửa, thư viện ảnh và các mục tham khảo. Các bảng không

bị bỏ đồng loạt vì bảng hội thoại của nhân vật có thể chứa lore quan trọng. Mỗi lần giữ hay loại một trang đều ghi lại lý do để kiểm tra sau này. Hiện giai đoạn làm sạch chính thức chưa hoàn thành; các phép đo trong báo cáo dùng bộ đo thử chỉ để khảo sát cấu trúc.

Chiến lược chia đoạn được đề xuất tại ADR-003 và đang chờ nhóm xem xét. Nội dung chính gồm: chia theo tiêu đề; mỗi chuỗi đưa vào mô hình embedding không quá 450 token, tính cả tiền tố và token đặc biệt; phần chồng lấn 50 token lấy từ cuối đoạn trước trong cùng một mục; đếm bằng tokenizer của chính mô hình embedding để không vượt giới hạn 512 token của mô hình [16]. Mỗi đoạn mở đầu bằng tiêu đề trang và đường dẫn tiêu đề, theo dạng sau.

```
passage: <tiêu đề trang>
<mục cha > mục con>
<nội dung đoạn>
```

Các quy tắc ranh giới gồm: không ghép nội dung khác trang, khác phạm vi hoặc khác loại nguồn chỉ để đủ độ dài; ưu tiên cắt tại ranh giới đoạn và câu; giữ lời thoại cùng người nói, giữ tiêu đề bảng cùng các hàng liên quan; và cho phép mục ngắn giữ nguyên thay vì ghép bừa. Để chọn giá trị khởi đầu, nhóm đo mô phỏng cửa sổ token trên mẫu 20 trang (Bảng 6). Mẫu được chọn trải đều theo thứ tự khám phá cân bằng danh mục, không phải mẫu thống kê đại diện cho toàn wiki, và chưa có truy vấn nào được chạy trong thí nghiệm này.

Bảng 6. Số cửa sổ chia đoạn theo giới hạn token trên mẫu 20 trang

Giới hạn tổng token	**Số cửa sổ**	**Phần thân dưới 300 token**	**Tiền tố không đủ chỗ**
350	141	112	0
400	131	102	0
450	127	101	0
500	125	100	0

Kết quả cho thấy tăng giới hạn từ 450 lên 500 token chỉ giảm thêm hai cửa sổ, trong khi 450 để lại khoảng trống cho tiêu đề và ngữ cảnh. Phần lớn cửa sổ có thân ngắn vì nhiều mục vốn ngắn; nhóm không kéo dài chúng bằng nội dung bổ sung. Cần nhấn mạnh rằng 450 và 50 là giá trị khởi đầu, chưa được chứng minh là tối ưu cho truy xuất; chúng cần được kiểm chứng bằng bộ câu hỏi chuẩn.

2.7. Chất lượng dữ liệu và giới hạn

Quá trình khảo sát và thu thập làm lộ nhiều vấn đề về chất lượng và pháp lý. Bảng 7 liệt kê các vấn đề, bằng chứng và cách xử lý hiện tại.

Bảng 7. Vấn đề chất lượng dữ liệu và quyết định xử lý

Vấn đề	**Bằng chứng**	**Xử lý**
Danh mục lẫn nội dung bản mở rộng	Danh mục chung chứa trang thuộc nhiều phạm vi	Nhận diện sơ bộ ở phần mở đầu và khung thông tin; giữ bản thô kèm lý do; trường phạm vi để trống và gắn cờ cần rà soát từng mục, không gán base hàng loạt
Không đối chiếu được với Fandom	robots.txt trả HTTP 403 cho cả 10 trang	Dừng và ghi lỗi; không dùng proxy; không kết luận hai nguồn giống hay khác
Truy vấn theo phiên bản bị từ chối	Một số yêu cầu `oldid` trả HTTP 403	Lấy theo tiêu đề và lưu số phiên bản thực tế trả về cùng HTML
Thư viện giải nén Brotli lỗi	Giải mã sai nội dung trong môi trường chạy	Yêu cầu mã hóa nội dung dạng identity
Ảnh khai báo sai định dạng	Một ảnh được khai báo WebP nhưng dữ liệu là PNG	Xác định bằng nội dung ảnh, lưu đuôi theo định dạng thật và ghi lại sai khác
Ảnh giao diện lọt ngưỡng kích thước	Biểu tượng đánh giá 45 px được giữ lại	Thêm quy tắc loại riêng, có kiểm thử, tính lại ngữ cảnh trên dữ liệu thô không cần tải lại
Giấy phép ảnh không rõ	Toàn bộ 1.328 lần xuất hiện ảnh thiếu thông tin giấy phép	Gắn nhãn cần rà soát; không suy ra giấy phép ảnh từ giấy phép bài viết; không tái phân phối
Mục tiêu 200 trang chưa dùng được cho thí nghiệm	Chưa có bước làm sạch và lập chỉ mục	Thí nghiệm cơ sở dùng tập con dữ liệu khác (mục 3.1)

Dữ liệu và ảnh thô có dung lượng lớn nên không đưa vào kho mã nguồn mà chuyển giữa các máy kèm mã băm kiểm tra. Số liệu thu thập phản ánh trạng thái của nguồn tại thời điểm tải, không phải trạng thái vĩnh viễn.

2.8. Lựa chọn thành phần mô hình

Hệ thống ghép nhiều thành phần; Bảng 8 nêu lựa chọn, lý do và mức độ đã kiểm chứng của từng thành phần.

Bảng 8. Lựa chọn thành phần của quy trình RAG

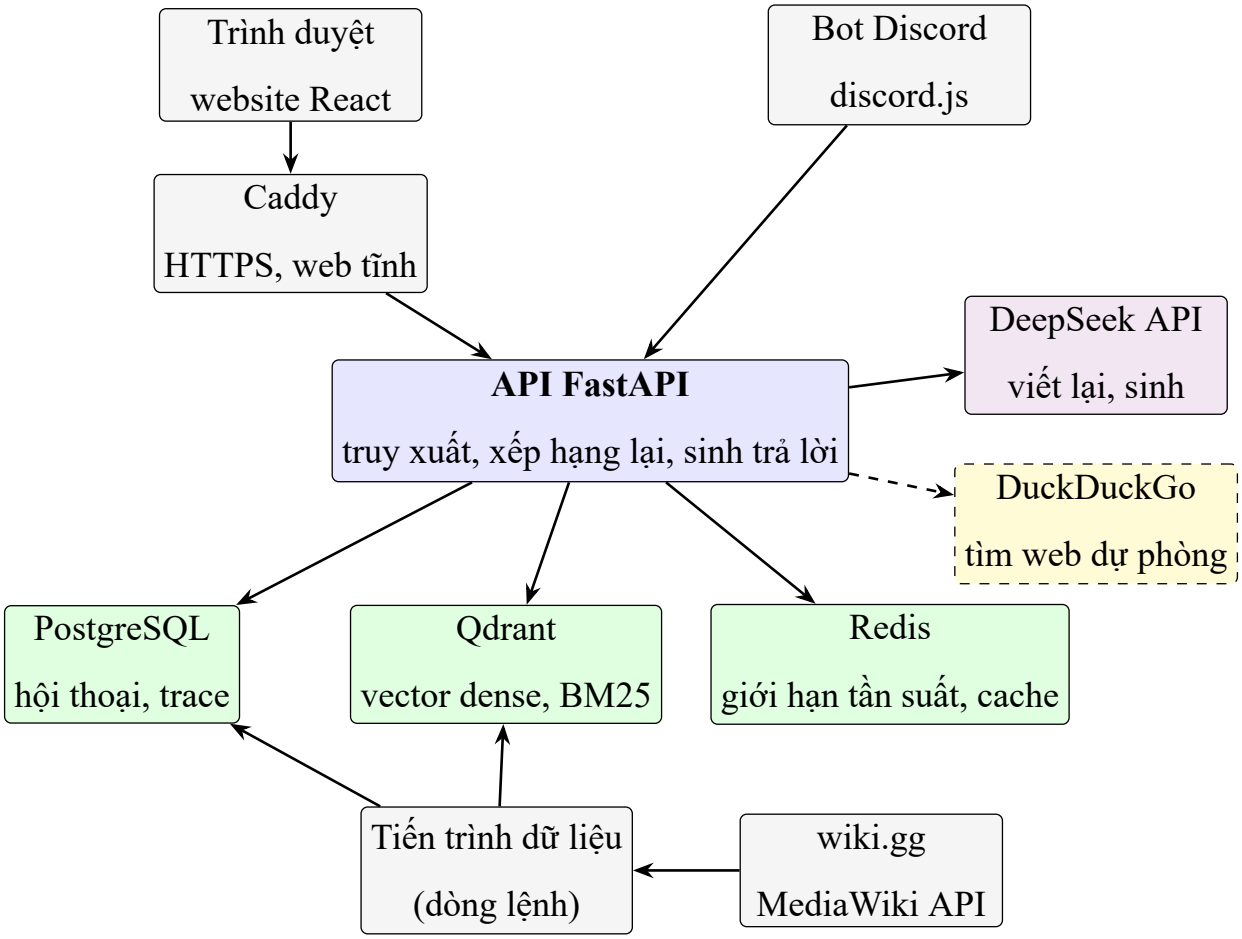
Thành phần	**Lựa chọn**	**Lý do**	**Mức kiểm chứng**
Embedding	`multilingual-e5-small`, 384 chiều, chạy bằng ONNX trên CPU	Hỗ trợ tiếng Việt và tiếng Anh, đủ nhỏ để chạy trên máy không GPU [4, 16]	Đã dùng trong thí nghiệm cơ sở
Từ khóa	BM25 dạng vector thưa trong Qdrant, bật hiệu chỉnh IDF	Bắt tên riêng chính xác; không cần thêm dịch vụ tìm kiếm riêng	Đã dùng trong thí nghiệm cơ sở
Hợp nhất	RRF qua truy vấn nhiều giai đoạn của Qdrant	Không cần chuẩn hóa điểm giữa hai hệ thống [5, 9]	Đã dùng trong thí nghiệm cơ sở
Xếp hạng lại	`bge-reranker-base` (ONNX); dự phòng `ms-marco-MiniLM-L-6-v2`	Cải thiện thứ hạng; mô hình nhỏ để chạy CPU [6, 18]	Đã chọn; chưa có số đo độ trễ trong kho mã; không dùng trong thí nghiệm cơ sở
Mô hình sinh	DeepSeek `deepseek-flash`, tắt chế độ suy luận	Giá thấp, ngữ cảnh dài, có đầu ra JSON và truyền theo luồng [10]	Bộ chuyển đổi đã viết; dùng trong thí nghiệm cơ sở
Chấm điểm ngoại tuyến	`deepseek-v4-pro`	Dùng mô hình khác mô hình sinh để chấm, tránh tự chấm bài của mình	Dự kiến; chưa triển khai
Tìm web dự phòng	DuckDuckGo với danh sách miền cho phép	Bổ sung khi kho thiếu bằng chứng	Thiết kế; chưa triển khai

Một số lựa chọn bị loại: chạy mô hình ngôn ngữ cục bộ vì máy chủ chỉ có bốn lõi CPU và sẽ chỉ đạt vài token mỗi giây; dùng khung điều phối lớn vì quy trình chỉ có vài bước và tự viết giúp dễ gỡ lỗi; cài đặt máy chủ tìm kiếm văn bản riêng vì Qdrant đã hỗ trợ vector thưa; và dùng thư viện học sâu đầy đủ khi chạy thật vì bản ONNX nhẹ và nhanh hơn trên CPU. Việc dùng mô hình sinh qua API làm dữ liệu câu hỏi và các đoạn nguồn được gửi ra dịch vụ bên ngoài; nhóm chỉ gửi nội dung công khai và không gửi thông tin định danh người dùng.

Về chi phí, với bảng giá của `deepseek-flash` (0,15 USD mỗi triệu token vào khi không trúng bộ nhớ đệm, 0,003 USD khi trúng, 0,60 USD mỗi triệu token ra, trong giờ thấp điểm; gấp đôi trong giờ cao điểm) [10], nếu một lượt hỏi dùng khoảng 4.000 token vào, trong đó khoảng 1.000 token là lời nhắc hệ thống được lưu đệm, và 500 token ra, chi phí xấp xỉ 0,00075 USD ở giờ thấp điểm và gấp đôi ở giờ cao điểm. Với giả định một phần ba lượt hỏi rơi vào giờ cao điểm, mức 1.000 câu mỗi ngày tốn khoảng 30 USD mỗi tháng. Đây là ước tính từ bảng giá và giả định độ dài, chưa phải số đo.

2.9. Thiết kế hệ thống

Hệ thống gồm phần phụ trợ chung, website, bot Discord và hạ tầng lưu trữ, chạy bằng Docker Compose trên một máy chủ (Hình 3). Web và bot chỉ là máy khách mỏng gọi cùng một giao diện lập trình nên logic hỏi đáp chỉ nằm ở một nơi.



Hình 3. Kiến trúc tổng thể của Index Beast

Giao diện lập trình được thiết kế gồm các điểm: kiểm tra sức khỏe, hỏi đáp trả về một lần, hỏi đáp truyền theo luồng (Server-Sent Events) để web hiển thị từng đoạn, gửi phản hồi, đọc lịch sử hội thoại và các điểm quản trị. Tại thời điểm báo cáo, phần phụ trợ mới triển khai điểm kiểm tra sức khỏe và một điểm thử gọi mô hình sinh dành cho môi trường phát triển; các điểm còn lại là hợp đồng dự kiến để web và bot phát triển song song. Điểm kiểm tra sức khỏe chạy song song các kiểm tra của từng dịch vụ, mỗi kiểm tra có giới hạn thời gian, và trả về 503 khi có dịch vụ lỗi để hệ thống giám sát phát hiện được. Các tài nguyên dùng chung (cơ sở dữ liệu, kho vector, bộ nhớ đệm, mô hình sinh) được đăng ký tại một nơi, nên thêm tài nguyên mới tự có trong kiểm tra sức khỏe và tự được đóng khi tắt ứng dụng.

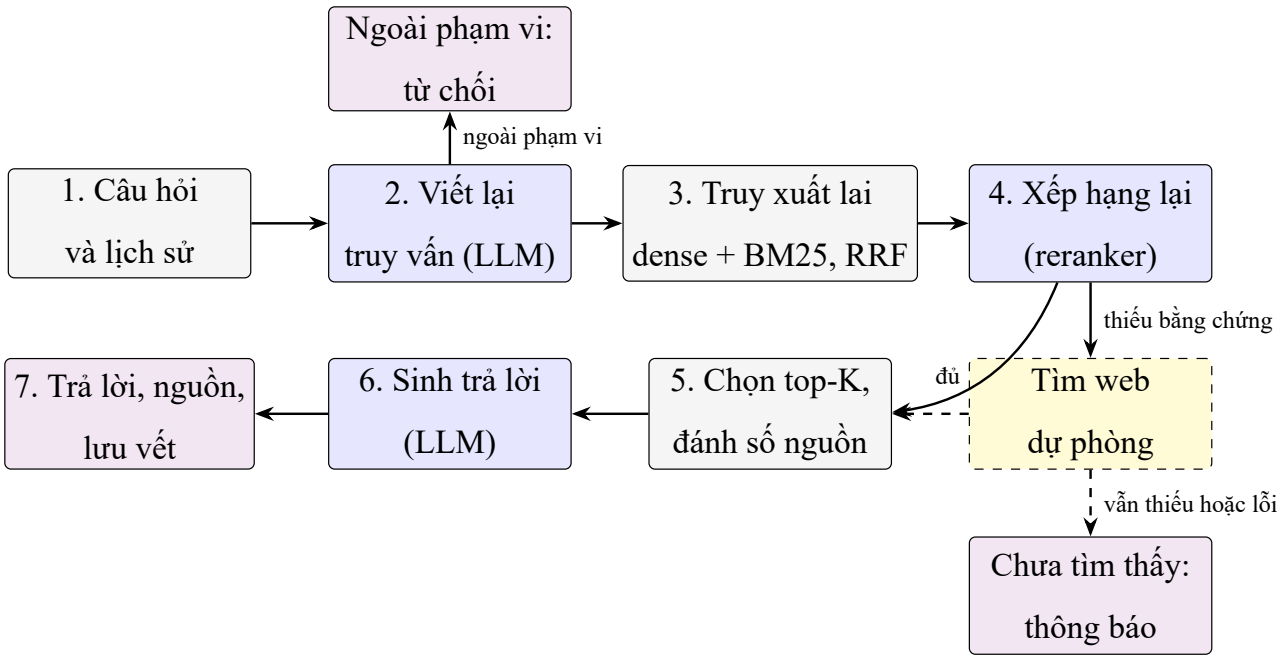
Bảng 9 nêu các công nghệ chính và phiên bản được cố định. Ngoài ra, dự án áp dụng quy ước không chạy tiến trình nào trên cổng mặc định của công cụ: cổng dự án bằng cổng gốc cộng 10.000, cụ thể API 18000, website 15173, PostgreSQL 15432, Qdrant 16333 và Redis 16379, mỗi cổng khai báo ở đúng một biến cấu hình. Ngoại lệ là Caddy dùng cổng 80 và 443 để trình duyệt và dịch vụ cấp chứng chỉ truy cập được.

Bảng 9. Công nghệ chính và phiên bản

Nhóm	**Công nghệ**	**Phiên bản**
Ngôn ngữ và khung phụ trợ	Python, FastAPI, Uvicorn, SQLAlchemy	3.11.9; 0.143.3; 0.54.0; 2.1.5
Chạy mô hình trên CPU	FastEmbed (ONNX Runtime), qdrant-client	0.9.0; 1.19.1
Lưu trữ	PostgreSQL, Qdrant, Redis	18.6; 1.19.2; 8.10.2
Giao diện	Node.js, TypeScript, React, Vite, Tailwind CSS	24.22.0; 6.0.3; 19.3.1; 8.3.4; 4.3.3
Bot và triển khai	discord.js, Caddy, Docker Compose	14.27.0; 2.11.7; Compose v2

2.10. Quy trình xử lý một câu hỏi

Hình 4 mô tả luồng xử lý khi người dùng đặt câu hỏi. Đây là thiết kế mục tiêu; thí nghiệm cơ sở ở Chương 3 mới thực hiện các bước truy xuất lai và sinh. Thí nghiệm đó chưa có bước viết lại hay dịch truy vấn, chưa xếp hạng lại và chưa tìm web: câu hỏi tiếng Việt và tiếng Anh được đưa thẳng vào cả hai hệ thống tìm kiếm.



Hình 4. Luồng xử lý một câu hỏi của hệ thống (thiết kế mục tiêu)

Các bước chính như sau. Ở bước viết lại, mô hình sinh nhận câu hỏi và lịch sử rồi trả về một đối tượng JSON gồm truy vấn tiếng Anh độc lập, ngôn ngữ người hỏi, danh sách thực thể và cờ ngoài phạm vi; nếu bước này lỗi, hệ thống dùng câu hỏi gốc. Ở bước truy xuất lai, truy vấn được mã hóa bằng embedding với tiền tố `query:`, đồng thời tìm bằng BM25, rồi hợp nhất bằng RRF trong một lần gọi đến Qdrant, có thể lọc theo phạm vi trò chơi. Bước xếp hạng lại giữ một số ít đoạn tốt nhất và so điểm cao nhất với một ngưỡng để quyết định có đủ bằng chứng không. Nếu không đủ và tính năng được bật, hệ thống tìm trên web nhưng chỉ nhận kết quả từ các miền cho phép, chia và xếp hạng lại các đoạn mới như dữ liệu trong kho, và đánh dấu câu trả lời dùng nguồn web. Nếu vẫn thiếu, hệ thống thông báo chưa tìm thấy và không gọi mô hình sinh, nhờ đó vừa tránh bịa vừa tiết kiệm chi phí.

Lời nhắc sinh câu trả lời tuân theo các quy tắc: chỉ dùng các đoạn được cung cấp; trích dẫn theo số của đoạn; trả lời bằng ngôn ngữ người hỏi nhưng giữ tên riêng gốc tiếng Anh; nêu rõ khi nội dung là diễn giải; trả lời ngắn, đi thẳng vào câu hỏi; và bỏ qua mọi chỉ dẫn nằm trong ngữ cảnh. Đáp án chuẩn của bộ câu hỏi không bao giờ được đưa vào chỉ mục hay lời nhắc.

2.11. Thiết kế đánh giá

Bộ câu hỏi chuẩn phiên bản đầu tiên có 30 câu, 20 tiếng Việt và 10 tiếng Anh, mã từ `IB-GS-001` đến `IB-GS-030`. Mỗi câu gồm bảy trường: mã, câu hỏi, ngôn ngữ, đáp án tham chiếu, danh sách địa chỉ nguồn, nhóm và độ khó. Bảng 10 cho cơ cấu của bộ câu hỏi.

Bảng 10. Cơ cấu bộ câu hỏi chuẩn phiên bản 0

Nhóm	**Số câu**	**Mã**	**Ví dụ**
Nhân vật	6	001–006	“Blaidd là ai và phục vụ ai?”
Sự kiện	6	007–012	Kết cục trận Malenia và Radahn tại Aeonia
Địa điểm	6	013–018	Vùng và chủ nhân của Stormveil Castle
Đa bước	6	019–024	Cha mẹ và lời nguyền của Malenia và Miquella
Ngoài phạm vi	6	025–030	Yêu cầu giải thích cốt truyện Dark Souls III

Độ khó được gán là 12 câu dễ, 12 câu trung bình và 6 câu khó. Sáu câu đa bước có ít nhất hai địa chỉ nguồn. Sáu câu ngoài phạm vi có danh sách nguồn rỗng và được chấm bằng tỉ lệ từ chối đúng, không đưa vào Recall và MRR. Nhóm đặt ra các quy tắc sử dụng: chỉ gửi câu hỏi cho hệ thống; với câu lore, chấm theo ý nghĩa và độ đủ ý chứ không khớp từng chữ; độ bám nguồn phải được đối chiếu với ngữ cảnh thực sự được truy xuất; chuẩn hóa địa chỉ chuyển hướng trước khi tính chỉ số truy xuất; không dùng bộ chuẩn toàn dự án để kết luận về một chỉ mục chỉ có một phần dữ liệu; và lưu phiên bản, cấu hình, dữ liệu và mã commit cùng mỗi lượt đánh giá. Các câu trong bộ này được soạn dựa trên nội dung wiki do công cụ tìm kiếm lập chỉ mục, chưa được kiểm tra trực tiếp từng trang và chưa được thành viên khác kiểm tra chéo, nên đáp án cần được xem xét lại trước khi dùng làm căn cứ kết luận.

Thí nghiệm cơ sở dùng thang điểm 0, 1, 2 cho cả độ đúng và độ bám nguồn, cùng Hit@5 và MRR@5 cho truy xuất. Do thí nghiệm này khác phạm vi dữ liệu của bộ chuẩn nên được dùng riêng một bộ 10 câu (5 tiếng Việt, 5 tiếng Anh) về trang phục trong tập dữ liệu.

2.12. Quy trình phát triển và bảo đảm chất lượng

Dự án được thực hiện bởi bốn thành viên trong sáu sprint, mỗi sprint một tuần, từ 19/10/2026 đến 29/11/2026. Các thành viên phụ trách lần lượt phần phụ trợ và hạ tầng, dữ liệu, truy xuất và mô hình ngôn ngữ, và giao diện cùng bot. Bảng 11 tóm tắt mục tiêu từng sprint.

Bảng 11. Kế hoạch sáu sprint

Sprint	Thời gian	Mục tiêu
1	19/10–25/10	Nền móng: môi trường, CI, khung API, thu thập mẫu, thí nghiệm RAG cơ sở, bộ câu hỏi chuẩn, giao diện và bot khởi đầu
2	26/10–01/11	Hoàn thiện dữ liệu: làm sạch, chia đoạn, lập chỉ mục, hợp đồng API
3	02/11–08/11	Nối RAG từ đầu đến cuối: viết lại truy vấn, xếp hạng lại, sinh trả lời có trích dẫn, truyền theo luồng
4	09/11–15/11	Chất lượng: bộ chấm tự động, tối ưu truy xuất, bảo vệ API, tìm web dự phòng
5	16/11–22/11	Triển khai thật, kiểm thử tải, giám sát và sao lưu
6	23/11–29/11	Kiểm thử với người dùng, sửa lỗi, đóng băng tính năng, bàn giao

Về kỹ thuật, mọi thay đổi đi qua nhánh riêng và yêu cầu hợp nhất (pull request) về nhánh phát triển chung, có người rà soát phê duyệt trước khi hợp nhất, và thông điệp commit theo quy ước Conventional Commits. Trước mỗi commit, các móc kiểm tra chạy công cụ kiểm tra và định dạng mã, chặn khóa bí mật và tệp quá lớn. Quy trình tích hợp liên tục kiểm tra mã phụ trợ (lint, kiểm thử đơn vị, kiểm thử tích hợp với các dịch vụ thật), dựng website và kiểm tra kiểu của bot trên mọi yêu cầu hợp nhất. Cấu hình được gom vào một tệp môi trường ở gốc kho mã; ứng dụng ở chế độ sản xuất sẽ từ chối khởi động nếu còn giá trị mẫu, và mỗi container chỉ nhận các biến nó cần. Sau mỗi nhóm công việc, người thực hiện viết tài liệu bàn giao đặt cùng mã nguồn.

CHƯƠNG 3. THỬ NGHIỆM VÀ ĐÁNH GIÁ

3.1. Thiết lập thực nghiệm

Mục đích của thí nghiệm ở Sprint 1 là kiểm chứng toàn bộ chuỗi truy xuất lai rồi sinh câu trả lời có trích dẫn có chạy được hay không, và đo sơ bộ hành vi của chuỗi này; đây không phải phép đo chất lượng của hệ thống hoàn chỉnh. Kế hoạch ban đầu dùng 200 trang mẫu do nhóm dữ liệu thu thập. Vì các trang này chưa được làm sạch và chưa có chỉ mục, thí nghiệm dùng tập dữ liệu khác: 240 bản ghi đầu tiên của tệp `armors.csv`, tệp có 568 bản ghi tại thời điểm đọc (25/10/2026). Mỗi bản ghi mô tả một món trang phục với tên, loại, khối lượng và đoạn mô tả lấy từ trò chơi. Đây là một phép thử trên tập con, không thay thế việc chạy trên dữ liệu đã xử lý của nhóm dữ liệu. Bảng 12 nêu cấu hình.

Bảng 12. Cấu hình thí nghiệm cơ sở

Hạng mục	**Giá trị**
Dữ liệu	240 bản ghi đầu của `armors.csv` theo thứ tự tệp, không xáo trộn, không bỏ bản ghi
Đơn vị lập chỉ mục	Mỗi bản ghi là một tài liệu và một điểm trong Qdrant; nội dung truy xuất gồm tên, loại, khối lượng và mô tả
Embedding	`multilingual-e5-small` qua FastEmbed (ONNX), 384 chiều; tiền tố `passage:` và `query:` thêm đúng một lần; đầu vào dài nhất 123 token, không phải rút gọn
Từ khóa	BM25 tiếng Anh dạng vector thưa, bật hiệu chỉnh IDF
Hợp nhất	Truy vấn nhiều giai đoạn của Qdrant với RRF, tham số mặc định; lấy 5 kết quả đầu
Kho vector	Qdrant ở chế độ bộ nhớ, tạo mới cho mỗi lần chạy
Mô hình sinh	`deepseek-flash`, tắt chế độ suy luận, tối đa 512 token đầu ra
Chưa có trong thí nghiệm	Viết lại và dịch truy vấn, xếp hạng lại, lọc theo phạm vi, tìm web
Câu hỏi	10 câu về trang phục: 5 tiếng Việt và 5 tiếng Anh, chọn từ các bản ghi đã biết có trong tập
Đáp án tham chiếu	Lấy từ chính nội dung CSV, không đưa vào chỉ mục hay lời nhắc
Thư viện	fastembed 0.9.0, qdrant-client 1.19.1, openai 3.33.0, onnxruntime 1.31.0, tokenizers 0.23.3

Lời nhắc yêu cầu trả lời thẳng, thường từ một đến ba câu, đúng ngôn ngữ câu hỏi, giữ tên riêng gốc, dẫn nguồn theo số trong năm kết quả đầu, và giữ sắc thái như "được đồn là" của nguồn. Nguồn trích dẫn được ghi là tệp dữ liệu kèm số thứ tự bản ghi, không tự tạo địa chỉ trang wiki. Lượt chạy được chấm có mã `20261025T144625694379Z` và chạy ngày 25/10/2026. Thời gian lập vector cho 240 bản ghi là 4,6 giây trên máy phát triển.

Việc chấm điểm được thực hiện thủ công theo thang 0, 1, 2 cho độ đúng (trả lời đúng và đủ ý được hỏi) và độ bám nguồn (mọi khẳng định được hỗ trợ bởi đoạn đã truy xuất). Người chấm là một trợ lý trí tuệ nhân tạo hỗ trợ nhóm, đối chiếu câu hỏi, đáp án tham chiếu và các đoạn đã lưu mà không chạy lại mô hình. Đây không phải chấm điểm độc lập của người thứ hai hay của một mô hình khác, và việc đối chiếu chỉ dựa trên nội dung tệp dữ liệu chứ chưa so với trò chơi.

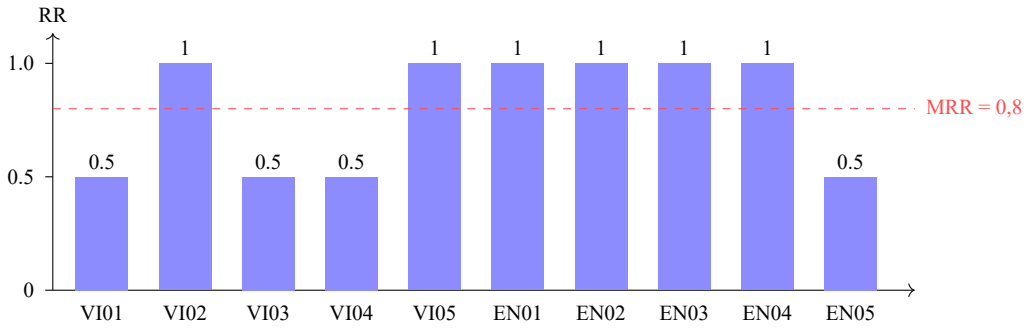
3.2. Kết quả truy xuất

Bảng 13 tổng hợp kết quả truy xuất. Một tài liệu được xem là đúng khi nó là bản ghi chứa câu trả lời của câu hỏi. Chỉ số trích dẫn hợp lệ nghĩa là các số hiệu trong câu trả lời trỏ đến đoạn có thật trong năm đoạn được đưa vào lời nhắc; chỉ số này chưa chứng minh câu trả lời được nguồn hỗ trợ.

Bảng 13. Kết quả truy xuất và sinh trả lời của thí nghiệm cơ sở

Ngôn ngữ	Số câu	Hit@5	MRR@5	Trích dẫn hợp lệ	Truy xuất (ms)	Sinh (ms)
Tiếng Việt	5	5/5	0,700	5/5	19,9	1.100,8
Tiếng Anh	5	5/5	0,900	5/5	17,4	995,2
Chung	**10**	**10/10**	**0,800**	**10/10**	—	—

Độ trễ là trung bình do notebook ghi lại trong một lượt chạy duy nhất trên máy phát triển, với Qdrant trong bộ nhớ và một tập chỉ 240 điểm. Con số này chỉ cho biết bậc độ lớn; nó không đại diện cho độ trễ khi chạy trên máy chủ thật với kho dữ liệu lớn hơn, bước viết lại và bước xếp hạng lại.



Hình 5. Nghịch đảo thứ hạng (RR) của tài liệu đúng đầu tiên cho từng câu hỏi

Cả 10 câu đều có tài liệu đúng trong năm kết quả đầu. Với bốn câu (VI01, VI03, VI04 và EN05) tài liệu đúng nằm ở hạng hai, còn hạng một là một biến thể hoặc một món khác trong cùng bộ. Ví dụ, với câu hỏi về chủ nhân của All-knowing Armor, hạng một là bản "altered" chỉ có mô tả "phiên bản nhẹ hơn", còn bản gốc chứa tên Gideon Ofnir nằm ở hạng hai. Các bản ghi cùng tên bộ giáp có vector và từ khóa giống nhau nên dễ chiếm các vị trí đầu. Mô hình sinh vẫn chọn được nguồn đúng trong số năm đoạn và trích dẫn nó, nhưng nếu giảm số đoạn đưa vào lời nhắc, những câu này sẽ gặp rủi ro. Đây là chỗ bước xếp hạng lại có thể cải thiện.

Cần đọc kết quả này một cách thận trọng. Các câu hỏi tiếng Việt đều chứa nguyên tên tiếng Anh của món trang phục, và trong thí nghiệm này câu hỏi không được dịch trước khi đưa vào BM25, nên phần từ khóa có thể khớp nhờ tên riêng. Vì vậy kết quả không cho phép kết luận về khả năng truy xuất chéo ngôn ngữ đối với câu hỏi tiếng Việt không chứa tên tiếng Anh. Thí nghiệm cũng chưa chạy riêng từng thành phần (chỉ vector, chỉ từ khóa) nên chưa đo được đóng góp của việc kết hợp.

3.3. Đánh giá câu trả lời

Bảng 14 nêu điểm và nhận xét cho từng câu. Điểm cho mỗi tiêu chí tối đa là 2; tổng độ đúng là 20 trên 20 và tổng độ bám nguồn là 20 trên 20.

Bảng 14. Điểm và nhận xét theo từng câu hỏi (thang 0–2)

Câu	Hạng nguồn đúng	Đúng	Bám nguồn	Nhận xét
VI01	2	2	2	Đúng người mặc là Gideon Ofnir; trích đúng bản gốc, không nhầm sang bản altered ở hạng một
VI02	1	2	2	Đúng "đọc vận mệnh trong các vì sao"; thêm một câu về hậu duệ của glintstone sorcerers có trong nguồn nhưng không được hỏi
VI03	2	2	2	Đúng hiệu ứng che tiếng bước chân; cách dịch "làm nghẹt" chưa tự nhiên, nên dùng "làm giảm"
VI04	2	2	2	Đúng gia đình hoàng gia Carian; trích đúng Carian Knight Armor thay vì Gauntlets ở hạng một
VI05	1	2	2	Đúng nhóm quý tộc ở kinh đô; câu thứ hai về Shattering có trong nguồn nhưng thừa; cách dịch "undead" cần chính xác hơn
EN01	1	2	2	Đủ hai ý được hỏi: người mặc và việc run sợ khi nhìn vào ý chí của Queen Marika
EN02	1	2	2	Đúng khác biệt: bản altered nhẹ hơn (3,2 so với 4,1); trích đúng hai biến thể
EN03	1	2	2	Nêu đúng ý "the night sky no longer cradles fate"; thêm phần hậu duệ glintstone sorcerers không cần thiết
EN04	1	2	2	Giữ đúng sắc thái "được đồn là Numen", không biến tin đồn thành sự thật
EN05	2	2	2	Xác định đúng man-wolf Blaidd; thêm câu về Gauntlets không cần thiết

Kết quả cho thấy mô hình bám khá sát nguồn: không có khẳng định nào ngoài nội dung được truy xuất, các sắc thái như "được đồn là" được giữ, và khi không có đủ thông tin cho phần nào đó (câu EN02) mô hình ghi rõ các bản ghi không nêu thêm khác biệt. Mô hình cũng xử lý được cặp đa ngôn ngữ: câu hỏi tiếng Việt về nguồn tiếng Anh được trả lời bằng tiếng Việt với tên riêng giữ nguyên.

Điểm tuyệt đối 20/20 không nên được hiểu là chất lượng 100% của chatbot. Các câu hỏi được chọn từ những món đã biết có trong tập, không có câu không thể trả lời, không có câu cần ghép nhiều nguồn, và mỗi đáp án tham chiếu nằm gọn trong một đoạn mô tả ngắn. Người chấm không độc lập và nguồn chưa được kiểm chứng với trò chơi. Với mẫu 10 câu, một câu

sai làm điểm giảm 10 điểm phần trăm, nên không thể ước lượng đáng tin cậy tỉ lệ đúng của hệ thống.

3.4. Phân tích lỗi và khả năng khái quát hóa

Thí nghiệm không có câu trả lời sai, nhưng các quan sát ghi nhận được là:

- **Thông tin thừa (4 câu: VI02, VI05, EN03, EN05).** Mô hình thêm chi tiết đúng nguồn nhưng không được hỏi. Hướng xử lý là bổ sung vào lời nhắc yêu cầu chỉ trả lời khía cạnh được hỏi, rồi đánh giá lại để xem tác động.
- **Dịch chưa tự nhiên (VI03, VI05).** Cần kiểm tra ở quy mô lớn hơn vì chỉ vài câu bị ảnh hưởng.
- **Biến thể chiếm hạng đầu (4 câu).** Cần so sánh thêm xếp hạng lại và cách ghép tên chính xác.
- **Chưa có phép thử các tình huống khó:** câu không có đáp án, câu không chứa nguyên tên tài liệu, câu diễn đạt lại, câu dễ nhầm biến thể, câu cần ghép nhiều nguồn.

Về khả năng khái quát, kết quả hiện chỉ hợp lệ cho đúng tập 240 bản ghi này. Nội dung chính của dự án là văn xuôi dài trong các trang wiki, được chia đoạn theo tiêu đề; số lượng đoạn lớn hơn nhiều, độ dài đoạn khác, và nhiều đoạn có nội dung gần nhau. Các điều kiện đó chưa được thử, nên cần chạy lại trên dữ liệu đã làm sạch cùng bộ câu hỏi chuẩn 30 câu trước khi nêu kết luận về chất lượng.

3.5. Kiểm thử kỹ thuật và chất lượng phần mềm

Ngoài thí nghiệm về mô hình, nhóm kiểm tra chất lượng của phần mềm nền tảng. Bảng 15 tổng hợp các kiểm tra chạy lại trên nhánh phát triển ngày 26/10/2026.

Bảng 15. Kết quả kiểm tra phần mềm trên nhánh phát triển

Thành phần	Kiểm tra	Kết quả
Phụ trợ (backend)	Công cụ kiểm tra và định dạng mã (117 tệp)	Đạt
Phụ trợ (backend)	Kiểm thử đơn vị: cấu hình, nhật ký, middleware, kiểm tra sức khỏe, vòng đời tài nguyên, quy ước cổng	79 đạt
Phụ trợ (backend)	Kiểm thử tích hợp với PostgreSQL, Redis, Qdrant thật	Đạt trong tích hợp liên tục
Website	Định dạng, biên dịch; 8 kiểm thử về luồng SSE, mã hóa UTF-8, ngắt luồng, lỗi 429, nguồn lỗi, ngôn ngữ (Node 24.22.0)	Đạt; 8 đạt trên 8
Bot Discord	Kiểm tra kiểu và định dạng	Đạt
Tích hợp liên tục	Bốn lần chạy gần nhất trên nhánh phát triển	Thành công

Quá trình này cũng giúp phát hiện và sửa một số lỗi thiết kế: tích hợp liên tục từng báo lỗi ở mọi yêu cầu hợp nhất do dòng mô tả mã quá dài; tệp cấu hình môi trường có chú thích cuối dòng bị công cụ dựng container hiểu thành giá trị; middleware ghi mã định danh yêu cầu bị mất khi truyền theo luồng nên đã viết lại ở mức ASGI; nhóm thống nhất không chạy tiến trình nào trên cổng mặc định của công cụ nên đặt quy ước cổng riêng, có kiểm thử chặn tái phạm. Các kiểm thử trên xác nhận phần nền tảng hoạt động đúng thiết kế; chúng không đo chất lượng hỏi đáp.

3.6. Chương trình minh họa

Nhóm cung cấp hai ứng dụng khách và một điểm thử phần phụ trợ. Trạng thái của từng phần được nêu ở Bảng 16 để tránh nhầm giữa giao diện hoàn thiện và chức năng đã nối với dữ liệu thật.

Bảng 16. Các thành phần minh họa và trạng thái

Thành phần	Trạng thái tại thời điểm báo cáo
Website	Trang hỏi đáp với ô nhập, hiển thị Markdown, danh sách nguồn và hộp trích dẫn, lịch sử hội thoại của khách (lưu trong trình duyệt, tối đa 30 cuộc), nút phản hồi, trang giới thiệu nguồn và giấy phép, trang quản trị, chuyển ngôn ngữ Việt hoặc Anh, giao diện tối, thích ứng từ 360 px. Chế độ mặc định là xem thử với dữ liệu cố định có nhãn minh họa, không gọi mô hình. Chế độ kết nối API đọc luồng SSE thật nhưng phần phụ trợ chưa có điểm hỏi đáp
Bot Discord	Khung ứng dụng, đăng ký lệnh gạch chéo và lệnh kiểm tra `/ping`; các lệnh hỏi đáp mới là tệp khung
API phụ trợ	Điểm kiểm tra sức khỏe và điểm thử mô hình sinh (chỉ ở môi trường phát triển); tài liệu Swagger ở môi trường phát triển
Thí nghiệm RAG	Notebook cho phép chạy lại thí nghiệm cơ sở, kể cả bước gọi mô hình sinh nếu được bật

Để chạy các thành phần trên máy phát triển, cần khởi động hạ tầng bằng `docker compose up -d --wait postgres qdrant redis`, chạy API bằng lệnh `make api` (địa chỉ `http://127.0.0.1:18000/api/docs`) và chạy giao diện bằng `npm run dev` trong thư mục `web` (địa chỉ `http://localhost:15173`). Hướng dẫn đầy đủ nằm trong tệp README của kho mã [14].

3.7. Kết luận và hướng phát triển

Báo cáo trình bày thiết kế và kết quả giai đoạn đầu của Index Beast, hệ thống hỏi đáp lore Elden Ring dựa trên truy xuất lai và mô hình ngôn ngữ lớn. Kết quả đạt được gồm: một nền tảng phát triển có môi trường dựng lại được, kiểm tra tự động và cấu hình an toàn; quy trình thu thập dữ liệu ngoại tuyến đã tải thử 200 trang cùng 1.188 ảnh với kiểm tra toàn vẹn; chiến lược chia đoạn được đề xuất kèm số đo; bộ 30 câu hỏi chuẩn cho đánh giá; và một thí nghiệm cơ sở cho thấy chuỗi truy xuất lai rồi sinh có trích dẫn chạy được trên tập con dữ liệu, với Hit@5 là 10/10, MRR@5 là 0,8 và điểm 20/20 cho cả độ đúng lẫn độ bám nguồn trên 10 câu.

Những kết quả này có giới hạn rõ. Dữ liệu của thí nghiệm là 240 bản ghi dạng bảng chứ chưa phải văn xuôi đã làm sạch; chưa có bước viết lại truy vấn, xếp hạng lại hay tìm web; câu hỏi được chọn từ những mục đã biết và do một người chấm không độc lập; bộ câu hỏi chuẩn chưa được kiểm tra chéo; chưa so sánh các biến thể của quy trình; và các mục tiêu về độ trễ,

chi phí, Recall@5 và độ đúng trên bộ chuẩn đều chưa được đo. Phạm vi nội dung theo từng mục (bản gốc hay bản mở rộng) và giấy phép của ảnh cũng chưa được rà soát xong.

Hướng phát triển theo kế hoạch gồm các bước sau. Sprint 2 hoàn thiện làm sạch, chia đoạn và lập chỉ mục trên toàn bộ dữ liệu, đồng thời chốt hợp đồng dữ liệu và hợp đồng giao diện lập trình. Sprint 3 nối quy trình từ đầu đến cuối với viết lại truy vấn, xếp hạng lại, sinh trả lời có trích dẫn, truyền theo luồng, và đo Recall cùng MRR trên bộ chuẩn. Sprint 4 bổ sung bộ chấm tự động có đối chiếu với người chấm, tối ưu tham số truy xuất, chặn tấn công qua lời nhắc, giới hạn tần suất và tìm web dự phòng. Sprint 5 triển khai thật và kiểm thử tải theo các mục tiêu độ trễ. Sprint 6 kiểm thử với người dùng thật, sửa lỗi và bàn giao. Ở mỗi giai đoạn, kết quả cần được ghi cùng điều kiện đo, phiên bản dữ liệu và mã nguồn để người khác kiểm tra lại được.

TÀI LIỆU THAM KHẢO

[1] D. Jurafsky và J. H. Martin (2025), Speech and Language Processing, bản thảo lần 3. https://web.stanford.edu/~jurafsky/slp3/.

[2] C. D. Manning, P. Raghavan và H. Schütze (2008), Introduction to Information Retrieval, Cambridge University Press. https://nlp.stanford.edu/IR-book/.

[3] S. Robertson và H. Zaragoza (2009), The Probabilistic Relevance Framework: BM25 and Beyond, Foundations and Trends in Information Retrieval, 3(4), 333–389.

[4] L. Wang và cộng sự (2024), Multilingual E5 Text Embeddings: A Technical Report, arXiv:2402.05672.

[5] G. V. Cormack, C. L. A. Clarke và S. Büttcher (2009), Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods, Proceedings of SIGIR 2009, 758–759.

[6] R. Nogueira và K. Cho (2019), Passage Re-ranking with BERT, arXiv:1901.04085.

[7] P. Lewis và cộng sự (2020), Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, Advances in Neural Information Processing Systems 33.

[8] Z. Ji và cộng sự (2023), Survey of Hallucination in Natural Language Generation, ACM Computing Surveys, 55(12), Article 248.

[9] Qdrant, Hybrid and Multi-Stage Queries. https://qdrant.tech/documentation/search/hybrid-queries/ (truy cập tháng 10/2026).

[10] DeepSeek API Docs, Models & Pricing; Thinking Mode. https://api-docs.deepseek.com/quick_start/pricing/ và https://api-docs.deepseek.com/guides/thinking_mode/ (truy cập 17/10/2026).

[11] MediaWiki, API:Categorymembers và API:Parsing wikitext. https://www.mediawiki.org/wiki/API:Categorymembers.

[12] Creative Commons, Attribution-ShareAlike 4.0 International (CC BY-SA 4.0). https://creativecommons.org/licenses/by-sa/4.0/.

[13] Elden Ring Wiki (wiki.gg). https://eldenring.wiki.gg/.

[14] Nhóm 4 Hollows, Index Beast, kho mã nguồn và tài liệu dự án. https://github.com/HoanBuCon/Index-Beast.

[15] S. Es, J. James, L. Espinosa-Anke và S. Schockaert (2023), RAGAS: Automated Evaluation of Retrieval Augmented Generation, arXiv:2309.15217.

[16] Hugging Face, intfloat/multilingual-e5-small, thẻ mô hình. https://huggingface.co/intfloat/multilingual-e5-small.

[17] Qdrant, FastEmbed: supported models. https://qdrant.github.io/fastembed/examples/Supported_Models/.

[18] Hugging Face, BAAI/bge-reranker-base, thẻ mô hình. https://huggingface.co/BAAI/bge-reranker-base.